

# **CELEBAL TECHNOLOGIES**

**Celebal Excellence Internship**

## **PROJECT REPORT**

# **E-Commerce Order Analytics System Using Python and SQL**

**Submitted in Partial Fulfillment of the Requirements of the  
Celebal Excellence Internship**

---

### **Submitted By**

**Name:** Ritik Chanana

**College:** Maharishi Markandeshwar (Deemed to be University), Mullana

**Branch:** B.Tech Computer Science Engineering

---

### **Internship Organization**

**Celebal Technologies**

---

### **Duration**

May 2026 – July 2026

---

# CERTIFICATE

This is to certify that the project report entitled "**E-Commerce Order Analytics System Using Python and SQL**" has been successfully completed by **Ritik C.**, a student of **B.Tech Computer Science Engineering (Artificial Intelligence & Machine Learning)** at **Maharishi Markandeshwar (Deemed to be University), Mullana**, as part of the **Celebal Technologies Summer Internship Program 2026**.

The project demonstrates the design and development of a complete data analytics system for processing and analyzing e-commerce order data using **Python, Pandas, SQL Server, and SQL**. The work includes realistic dataset generation with intentional inconsistencies, data cleaning and validation, database design, SQL-based analytical reporting, customer segmentation, cohort analysis, and the development of a command-line reporting tool.

This project has been completed with sincerity and dedication and represents the candidate's understanding and practical implementation of data analytics concepts acquired during the internship.

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Celebal Technologies** for providing me with the opportunity to participate in the **Summer Internship Program 2026**. This internship has been an invaluable learning experience that enabled me to strengthen my understanding of data analytics, database management, and software development through practical implementation.

I am deeply thankful to my internship mentors and the entire Celebal Technologies team for their continuous guidance, encouragement, and support throughout the project. Their valuable insights and constructive feedback helped me overcome challenges and improve both my technical knowledge and problem-solving skills.

I would also like to extend my heartfelt appreciation to the faculty members of **Maharishi Markandeshwar (Deemed to be University), Mullana**, for providing the academic foundation and motivation necessary to successfully complete this project. Their constant encouragement played a significant role in enhancing my confidence and technical abilities.

This project, titled "**E-Commerce Order Analytics System Using Python and SQL**," provided me with hands-on experience in designing and developing a complete data analytics pipeline. The project involved generating realistic e-commerce datasets with intentional inconsistencies, performing data cleaning and validation using Python and Pandas, designing a relational database in SQL Server, implementing advanced SQL analytics using joins, aggregations, Common Table Expressions (CTEs), window functions, cohort analysis, and customer segmentation, and developing a command-line reporting tool to generate dynamic business reports. Working on each phase of the project significantly enhanced my understanding of real-world data engineering and analytics workflows.

Finally, I would like to express my gratitude to my parents, teachers, and friends for their continuous encouragement, motivation, and unwavering support throughout the internship. Their belief in my abilities inspired me to complete this project with dedication and enthusiasm.

I am grateful to everyone who contributed directly or indirectly to the successful completion of this internship project. The knowledge, experience, and confidence gained during this journey will remain valuable throughout my academic and professional career.

# TABLE OF CONTENTS

1. Abstract
2. Chapter 1: Introduction
  - 1.1 Project Overview
  - 1.2 Objectives of the Project
  - 1.3 Scope of the Project
3. Chapter 2: System Design
  - 2.1 System Architecture
  - 2.2 Project Folder Structure
  - 2.3 Technologies Used
  - 2.4 Database Design
4. Chapter 3: Implementation
  - 3.1 Dataset Generation
  - 3.2 Data Cleaning and Validation
  - 3.3 Database Creation and Data Loading
  - 3.4 SQL Analytics
  - 3.5 Customer Segmentation
  - 3.6 Cohort and Retention Analysis
  - 3.7 Command-Line Reporting Tool
5. Chapter 4: Results and Analysis
  - 4.1 Dataset Summary
  - 4.2 SQL Query Results
  - 4.3 Revenue Analysis
  - 4.4 Customer Insights
  - 4.5 CLI Report Outputs
6. Chapter 5: Conclusion and Future Scope
  - 5.1 Conclusion
  - 5.2 Future Scope

# ABSTRACT

The **E-Commerce Order Analytics System Using Python and SQL** is a comprehensive data analytics project developed to demonstrate the complete lifecycle of processing, managing, and analyzing e-commerce order data. The primary objective of this project is to build an end-to-end analytics system that begins with realistic data generation and extends to advanced business reporting through SQL-based analysis and a command-line reporting tool.

The project starts by generating four interconnected datasets—**Customers, Products, Orders, and Order Items**—using Python libraries such as **Faker, Pandas, and Random**. To simulate real-world business scenarios, intentional data inconsistencies including duplicate records, missing values, invalid email addresses, malformed date formats, broken foreign key references, and negative quantities were introduced into the datasets. These inconsistencies provide a realistic environment for performing data cleaning and validation.

The generated datasets were cleaned and transformed using the **Pandas** library. Various preprocessing techniques such as handling missing values, removing duplicate records, correcting data types, validating referential integrity, and standardizing date formats were implemented to ensure high-quality data suitable for database storage and analysis.

The cleaned datasets were then loaded into **Microsoft SQL Server**, where a relational database schema was created using appropriate primary keys, foreign keys, and integrity constraints. Advanced SQL techniques including **JOINS, aggregate functions, Common Table Expressions (CTEs), window functions, ranking functions, and cohort analysis** were implemented to derive meaningful business insights. These analyses included customer lifetime value, monthly revenue trends, product performance, average order value, customer segmentation using RFM metrics, retention analysis, and revenue growth calculations.

To improve usability, a Python-based **Command-Line Interface (CLI)** reporting tool was developed. The tool allows users to generate multiple business reports dynamically by executing SQL queries through simple command-line arguments. Robust error handling and input validation were incorporated to ensure reliable operation under different scenarios.

Overall, this project demonstrates the practical application of Python programming, data preprocessing, relational database design, SQL analytics, and business intelligence techniques in developing a complete e-commerce analytics solution. The experience gained through this project provides a strong foundation in data engineering, data analytics, and database management while highlighting the importance of data quality, structured database design, and analytical decision-making in modern business environments.

# CHAPTER 1: INTRODUCTION

## 1.1 Project Overview

In today's digital economy, e-commerce platforms generate massive volumes of transactional data every day. This data contains valuable information about customer behavior, product performance, sales trends, and business operations. However, raw transactional data often contains inconsistencies such as duplicate records, missing values, incorrect formats, and invalid relationships, making it unsuitable for direct analysis. Therefore, effective data generation, cleaning, validation, and analysis are essential for extracting meaningful business insights and supporting informed decision-making.

The **E-Commerce Order Analytics System Using Python and SQL** is an end-to-end data analytics solution designed to simulate a real-world e-commerce environment. The project demonstrates the complete data analytics workflow, beginning with the generation of realistic datasets and ending with advanced business reporting. Four interconnected datasets—Customers, Products, Orders, and Order Items—were generated using Python while intentionally introducing data quality issues such as duplicate records, missing values, invalid email addresses, malformed date formats, negative quantities, and broken foreign key references. These inconsistencies closely resemble real-world data challenges and provide an opportunity to apply practical data cleaning techniques.

After generating the datasets, Python and the Pandas library were used to clean, validate, and standardize the data by removing duplicate records, handling missing values, correcting data types, validating relationships among tables, and ensuring consistency across datasets. The cleaned data was then loaded into Microsoft SQL Server, where a relational database schema was designed using appropriate primary key and foreign key constraints to maintain data integrity.

Advanced SQL queries were implemented to perform comprehensive business analytics, including revenue analysis, customer lifetime value calculation, product performance evaluation, customer segmentation, cohort analysis, retention analysis, ranking, and trend analysis using JOINS, aggregate functions, Common Table Expressions (CTEs), and window functions. Finally, a Python-based Command-Line Interface (CLI) reporting tool was developed to generate dynamic reports directly from the SQL database, providing an efficient and user-friendly way to access business insights.

The project demonstrates the practical integration of Python, Pandas, SQL Server, and SQL analytics techniques to build a complete data analytics pipeline. It reflects real-world data engineering practices and highlights the importance of data quality, relational database design, and analytical reporting in modern business intelligence systems.

## 1.2 Objectives of the Project

The primary objective of this project is to design and develop a complete e-commerce data analytics system that demonstrates the practical application of Python, SQL, and database technologies in solving real-world business problems. The project focuses on generating realistic datasets, maintaining data quality, implementing advanced SQL analytics, and developing an efficient reporting system capable of providing valuable business insights.

The specific objectives of the project are as follows:

1. To generate realistic e-commerce datasets consisting of Customers, Products, Orders, and Order Items using Python libraries such as Faker, Random, and Pandas.
2. To simulate real-world data quality issues by intentionally introducing duplicate records, missing values, invalid email addresses, malformed date formats, broken foreign key references, and other inconsistencies within the generated datasets.
3. To perform comprehensive data cleaning and preprocessing using the Pandas library by handling missing values, removing duplicate records, correcting data types, validating referential integrity, and standardizing the overall dataset.
4. To design and implement a relational database in Microsoft SQL Server using appropriate primary key, foreign key, and integrity constraints for efficient data storage and management.
5. To perform advanced SQL-based business analytics using JOIN operations, aggregate functions, Common Table Expressions (CTEs), window functions, ranking functions, and subqueries.
6. To analyze key business metrics such as customer lifetime value, monthly revenue trends, product performance, average order value, and sales growth.
7. To perform customer segmentation using purchase frequency, spending behavior, and RFM (Recency, Frequency, Monetary) analysis for improved customer understanding.
8. To implement cohort analysis and customer retention analysis in order to evaluate customer engagement and repeat purchasing behavior over time.
9. To develop a Python-based Command-Line Interface (CLI) reporting tool capable of generating dynamic business reports directly from the SQL database while ensuring proper input validation and error handling.
10. To gain practical experience in data engineering, relational database management, SQL analytics, and business intelligence by building a complete end-to-end analytics pipeline that reflects real-world industry practices.

## 1.3 Scope of the Project

The scope of the **E-Commerce Order Analytics System Using Python and SQL** is to develop a complete data analytics solution capable of generating, processing, storing, and analyzing e-commerce transaction data. The project covers the entire data lifecycle, beginning with realistic dataset generation and ending with business intelligence reporting through SQL analytics and a Python-based command-line interface.

The system is designed to generate realistic datasets for customers, products, orders, and order items while intentionally introducing common data quality issues such as duplicate records, missing values, invalid email addresses, malformed date formats, inconsistent records, and broken foreign key relationships. These inconsistencies simulate real-world scenarios and provide an opportunity to perform practical data cleaning and validation using the Pandas library.

The project also includes the design and implementation of a relational database using Microsoft SQL Server. Cleaned datasets are imported into the database, where primary keys, foreign keys, and integrity constraints are applied to maintain consistency and reliability. Advanced SQL techniques including JOIN operations, aggregate functions, Common Table Expressions (CTEs), window functions, ranking functions, and cohort analysis are implemented to generate meaningful business insights from transactional data.

In addition, the project focuses on analyzing important business metrics such as total revenue, monthly sales trends, average order value, customer lifetime value, product performance, customer retention, purchase frequency, and customer segmentation using RFM (Recency, Frequency, Monetary) analysis. A Python-based Command-Line Interface (CLI) reporting tool is developed to allow users to generate dynamic reports by executing predefined SQL queries with appropriate input validation and error handling.

The current implementation is designed for educational and analytical purposes using simulated datasets. While the system effectively demonstrates real-world data engineering and analytics concepts, it does not include features such as real-time transaction processing, web-based dashboards, online payment integration, authentication mechanisms, or cloud-based deployment. These features can be incorporated in future versions to transform the project into a production-ready business intelligence solution.

Overall, the project provides a strong foundation in Python programming, data preprocessing, SQL database management, and business analytics while demonstrating industry-standard practices used in modern e-commerce analytics systems.



# CHAPTER 2: SYSTEM DESIGN

## 2.1 System Architecture

The E-Commerce Order Analytics System follows a modular architecture that separates each stage of the data analytics pipeline into independent components. This approach improves maintainability, scalability, and code organization while allowing each module to perform a specific responsibility within the overall workflow.

The system begins with the generation of realistic e-commerce datasets using Python. Four interconnected datasets—Customers, Products, Orders, and Order Items—are created with the help of the Faker, Random, and Pandas libraries. To simulate real-world business scenarios, intentional inconsistencies such as duplicate records, missing values, invalid email addresses, malformed date formats, negative quantities, and broken foreign key references are introduced into the raw datasets.

The generated datasets are then processed using a dedicated data cleaning module developed with the Pandas library. During this stage, duplicate records are removed, missing values are handled appropriately, incorrect data formats are standardized, invalid records are identified, and referential integrity between related datasets is validated. The cleaned datasets are subsequently exported for database loading.

After preprocessing, the cleaned CSV files are imported into Microsoft SQL Server. A relational database schema consisting of Customers, Products, Orders, and Order Items tables is created using appropriate primary keys, foreign keys, and integrity constraints. This ensures data consistency while supporting efficient querying and analysis.

The SQL analytics module performs various business intelligence operations by executing advanced SQL queries. These include JOIN operations, aggregate functions, Common Table Expressions (CTEs), window functions, ranking functions, cohort analysis, customer retention analysis, and RFM-based customer segmentation. These queries generate valuable insights related to revenue trends, customer purchasing behavior, product performance, and sales growth.

Finally, a Python-based Command-Line Interface (CLI) reporting tool connects to the SQL Server database using the PyODBC library. Based on user-specified command-line arguments, the tool dynamically executes SQL queries and presents formatted business reports in tabular form using the Tabulate library. The CLI also incorporates robust error handling, database connection validation, and user input validation to ensure reliable system performance.

The modular architecture adopted in this project ensures that each phase—from data generation to business reporting—operates independently while contributing to a unified end-to-end analytics workflow. This design reflects industry-standard data engineering practices and enables easy extension of the system with additional analytical modules in the future.

+-----+  
| Python Data Generator |  
| (Faker + Random + Pandas)|

+-----+-----+  
|  
v

+-----+  
| Raw CSV Datasets |  
| Customers | Products |  
| Orders | Order Items |

+-----+-----+  
|  
v

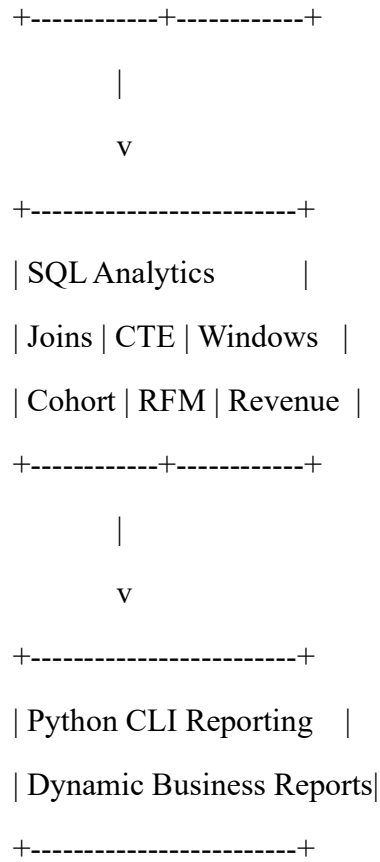
+-----+  
| Data Cleaning & |  
| Validation (Pandas) |

+-----+-----+  
|  
v

+-----+  
| Cleaned CSV Datasets |

+-----+-----+  
|  
v

+-----+  
| Microsoft SQL Server |  
| Database & Schema |



## 2.2 Project Folder Structure

A well-organized project structure is essential for improving code readability, simplifying maintenance, and enabling efficient collaboration. The E-Commerce Order Analytics System follows a modular directory structure where datasets, Python scripts, SQL scripts, generated reports, and project documentation are stored in separate folders. This organization ensures that each component of the project is easy to locate, modify, and manage throughout the development lifecycle.

The project directory is organized as follows:

ecommerce-analytics-system/

```

|
|
├── data/
|   ├── raw/
|       ├── customers.csv

```

```
| | |— products.csv
| | |— orders.csv
| | |— order_items.csv
| |
| |— cleaned/
| |   |— customers_clean.csv
| |   |— products_clean.csv
| |   |— orders_clean.csv
| |   |— order_items_clean.csv
|
|— scripts/
|   |— generate_data.py
|   |— clean_data.py
|   |— report_cli.py
|
|— sql/
|   |— schema.sql
|   |— aggregations.sql
|   |— window_functions.sql
|   |— cohort_analysis.sql
|
|— output/
|   |— sample_reports/
|
|— README.md
|
|— requirements.txt
```

The **data** directory contains all datasets used throughout the project. The **raw** folder stores the original datasets generated by the Python data generation script, including intentionally introduced inconsistencies such as duplicate records, missing values, malformed dates, and invalid references. The **cleaned** folder contains the processed datasets after data cleaning, validation, and preprocessing using the Pandas library.

The **scripts** directory contains the core Python programs responsible for different stages of the project. The `generate_data.py` script creates realistic e-commerce datasets with controlled inconsistencies. The `clean_data.py` script performs preprocessing tasks such as duplicate removal, missing value handling, data validation, and referential integrity checks. The `report_cli.py` script provides a command-line interface that connects to the SQL Server database and generates analytical reports based on user-selected options.

The **sql** directory stores all SQL scripts required for database creation and analytics. The `schema.sql` file creates the relational database schema with primary keys, foreign keys, and constraints. Additional SQL files contain queries for joins, aggregations, window functions, customer segmentation, and cohort analysis used to derive business insights from the cleaned data.

The **output** directory stores generated reports and sample outputs produced during project execution. These reports demonstrate the results of SQL analytics and the functionality of the command-line reporting tool.

The **README.md** file provides comprehensive documentation describing the project architecture, software requirements, installation steps, execution procedure, and usage instructions. The **requirements.txt** file lists all Python libraries required to execute the project successfully, allowing the environment to be recreated easily on another system.

This structured organization improves project maintainability, promotes modular development, and follows software engineering best practices commonly adopted in data engineering and analytics projects.

## 2.3 Technologies Used

The development of the **E-Commerce Order Analytics System Using Python and SQL** involved the integration of various programming languages, libraries, database technologies, and development tools. Each technology played a specific role in different stages of the project, including dataset generation, data preprocessing, database management, SQL analytics, and report generation. The selection of these technologies ensured that the system was efficient, modular, and capable of performing end-to-end data analysis.

The following table presents the technologies used during the development of the project along with their respective purposes.

| <b>Technology / Tool</b>            | <b>Purpose in the Project</b>                                                                                                                                                      |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Python 3.11</b>                  | Primary programming language used for dataset generation, data cleaning, database connectivity, and command-line application development.                                          |
| <b>Pandas</b>                       | Used for reading CSV files, cleaning datasets, handling missing values, removing duplicate records, validating data, and exporting cleaned datasets.                               |
| <b>Faker</b>                        | Generated realistic customer information, including names, email addresses, and registration dates for simulating real-world e-commerce data.                                      |
| <b>Random Module</b>                | Generated random values for products, prices, quantities, customer types, payment methods, discounts, and order details.                                                           |
| <b>Microsoft SQL Server</b>         | Stored the cleaned datasets and performed advanced SQL-based data analysis using relational database concepts.                                                                     |
| <b>SQL Management Studio (SSMS)</b> | Used to create database schemas, execute SQL queries, manage tables, and validate query results.                                                                                   |
| <b>SQL</b>                          | Implemented table creation, joins, aggregate functions, Common Table Expressions (CTEs), window functions, cohort analysis, customer segmentation, and business reporting queries. |
| <b>PyODBC</b>                       | Connected the Python application to Microsoft SQL Server, allowing SQL queries to be executed directly from the command-line reporting tool.                                       |
| <b>Tabulate</b>                     | Displayed SQL query results in a structured tabular format within the command-line interface.                                                                                      |
| <b>Visual Studio Code</b>           | Served as the primary Integrated Development Environment (IDE) for writing Python scripts, SQL files, and project documentation.                                                   |
| <b>GitHub</b>                       | Used for project version control, source code management, and final project submission.                                                                                            |
| <b>CSV Files</b>                    | Used as an intermediate storage format for raw and cleaned datasets before importing them into the SQL Server database.                                                            |

|                          |                               |
|--------------------------|-------------------------------|
| <b>Technology / Tool</b> | <b>Purpose in the Project</b> |
|--------------------------|-------------------------------|

|                           |                                                                                                                                                  |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Windows PowerShell</b> | Executed Python scripts, managed the virtual environment, installed dependencies, and tested the command-line reporting tool during development. |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|

The integration of these technologies enabled the successful development of a complete e-commerce analytics pipeline. Python and its supporting libraries were used for data generation and preprocessing, while Microsoft SQL Server provided a reliable relational database environment for storing and analyzing structured data. SQL was utilized to perform advanced analytical operations and generate meaningful business insights. The combination of these tools resulted in a scalable, modular, and efficient system capable of handling the complete workflow from raw data generation to analytical reporting.

## 2.4 Database Design

The E-Commerce Order Analytics System is built on a relational database model implemented using Microsoft SQL Server. The database is designed to efficiently store, manage, and analyze e-commerce transaction data while maintaining data consistency and integrity through the use of primary keys, foreign keys, and relational constraints.

The database consists of four interconnected tables: **Customers**, **Products**, **Orders**, and **Order\_Items**. These tables are logically related to represent the complete order processing workflow of an e-commerce platform. The relational design minimizes data redundancy, ensures referential integrity, and supports efficient execution of complex analytical queries.

### Customers Table

The **Customers** table stores the personal and account-related information of registered customers. Each customer is uniquely identified using a **Customer ID**, which serves as the primary key. Additional information such as customer name, email address, registration date, and customer type is maintained to support customer segmentation and behavioral analysis.

**Primary Key:** Customer\_ID

**Attributes:**

- Customer\_ID
- Customer\_Name
- Email
- Registration\_Date

- Customer\_Type
- 

### Products Table

The **Products** table contains detailed information about products available for purchase. Every product is assigned a unique **Product ID**, which acts as the primary key. The table also stores product category, subcategory, cost price, selling price, and stock quantity, enabling product-level sales analysis and inventory evaluation.

**Primary Key:** Product\_ID

**Attributes:**

- Product\_ID
  - Product\_Name
  - Category
  - Subcategory
  - Cost\_Price
  - Selling\_Price
  - Stock\_Quantity
- 

### Orders Table

The **Orders** table records every customer order placed on the platform. Each order is identified using a unique **Order ID** and is associated with a customer through the **Customer ID** foreign key. The table also stores order date, payment method, shipping method, order status, and region code.

**Primary Key:** Order\_ID

**Foreign Key:** Customer\_ID → Customers(Customer\_ID)

**Attributes:**

- Order\_ID
- Customer\_ID
- Order\_Date
- Payment\_Method
- Shipping\_Method



- Order\_Status
  - Region\_Code
- 

### **Order\_Items Table**

The **Order\_Items** table stores the individual products included in each order. Since a single order may contain multiple products, this table establishes a many-to-one relationship with both the Orders and Products tables. Each record represents one product purchased in a specific order and contains pricing and discount information required for revenue analysis.

**Primary Key:** Item\_ID

**Foreign Keys:**

- Order\_ID → Orders(Order\_ID)
- Product\_ID → Products(Product\_ID)

**Attributes:**

- Item\_ID
  - Order\_ID
  - Product\_ID
  - Quantity
  - Unit\_Price
  - Discount\_Percentage
  - Discount\_Amount
  - Final\_Price
- 

### **Relationship Between Tables**

The relational database follows a normalized structure to eliminate redundancy and maintain data consistency.

- One customer can place multiple orders.
- One order can contain multiple products.
- One product can appear in multiple orders.

- The Order\_Items table acts as a bridge between the Orders and Products tables by storing transaction-level information for each purchased product.

This relational design supports efficient implementation of SQL joins, aggregate queries, customer segmentation, cohort analysis, and business intelligence reporting.

### Database Schema Summary

| Table Name | Primary Key | Foreign Key(s) |
|------------|-------------|----------------|
|------------|-------------|----------------|

|           |             |   |
|-----------|-------------|---|
| Customers | Customer_ID | — |
|-----------|-------------|---|

|          |            |   |
|----------|------------|---|
| Products | Product_ID | — |
|----------|------------|---|

|        |          |             |
|--------|----------|-------------|
| Orders | Order_ID | Customer_ID |
|--------|----------|-------------|

|             |         |                      |
|-------------|---------|----------------------|
| Order_Items | Item_ID | Order_ID, Product_ID |
|-------------|---------|----------------------|

The database schema was designed according to relational database principles to ensure data integrity, minimize redundancy, and support efficient analytical operations. The use of primary keys uniquely identifies each record, while foreign key relationships maintain consistency across interconnected tables. This structured design enables the execution of complex SQL queries involving joins, aggregations, window functions, Common Table Expressions (CTEs), customer segmentation, and cohort analysis, thereby providing meaningful business insights from the e-commerce datasets.

# CHAPTER 3: IMPLEMENTATION

## 3.1 Dataset Generation

The first phase of the project involved generating realistic e-commerce datasets using Python. The objective of this phase was to simulate the operational data of an online retail platform while intentionally introducing common data quality issues that occur in real-world business environments. These datasets served as the foundation for the subsequent data cleaning, database implementation, and analytical reporting phases.

The dataset generation process was implemented in the `generate_data.py` script using the **Faker**, **Random**, and **Pandas** libraries. Faker was used to generate realistic customer information such as names, email addresses, and registration dates, while the Random module was used to generate product information, pricing, quantities, payment methods, discounts, order statuses, and other business-related attributes. Pandas was used to organize the generated records into structured DataFrames and export them as CSV files.

Four interconnected datasets were generated as part of this project:

### Customers Dataset

The Customers dataset contains information about registered customers. Each customer was assigned a unique Customer ID along with a name, email address, registration date, and customer type. To simulate real-world data quality problems, duplicate customer records, missing email addresses, and invalid email formats were intentionally introduced into the dataset.

### Products Dataset

The Products dataset stores information related to products available on the e-commerce platform. Each product contains details such as Product ID, product name, category, subcategory, cost price, selling price, and stock quantity. Realistic product categories were created to represent different sections of an online marketplace. Additional inconsistencies, including duplicate records, missing selling prices, irregular text formatting, and inconsistent capitalization, were deliberately added to test data cleaning techniques.

### Orders Dataset

The Orders dataset records customer purchases made on the platform. Each order is uniquely identified using an Order ID and is linked to a customer through the Customer ID field. The dataset includes attributes such as order date, payment method, shipping method, order status, and region code. To simulate operational challenges, duplicate orders, null customer IDs, malformed date formats, and future dates were intentionally inserted into selected records.

## Order Items Dataset

The Order Items dataset contains transaction-level information describing the products included in each order. Each record stores the Order ID, Product ID, quantity purchased, unit price, discount percentage, discount amount, and final price after discount. The dataset also includes intentionally generated anomalies such as duplicate records, negative quantities, invalid order references, and invalid product references to simulate broken referential integrity and inaccurate transaction data.

## Dataset Summary

The final generated datasets consist of realistic e-commerce records with controlled inconsistencies, enabling comprehensive testing of data cleaning and validation techniques in later stages of the project.

| Dataset     | Original Records | Intentional Inconsistencies Introduced                                         |
|-------------|------------------|--------------------------------------------------------------------------------|
| Customers   | 500              | Duplicate records, invalid emails, missing emails                              |
| Products    | 500              | Duplicate records, missing selling prices, inconsistent text formatting        |
| Orders      | 1000             | Duplicate records, null customer IDs, malformed dates, future dates            |
| Order Items | 4000             | Duplicate records, negative quantities, invalid Order IDs, invalid Product IDs |

The generated datasets were exported as CSV files and stored in the **data/raw/** directory. These raw datasets intentionally contained data quality issues so that realistic preprocessing and validation techniques could be demonstrated during the next phase of the project. This approach closely reflects real-world data engineering workflows, where incoming data is rarely perfect and must be cleaned before being used for business analysis or decision-making.

## 3.2 Data Cleaning and Validation

The raw datasets generated during the previous phase intentionally contained several data quality issues such as duplicate records, missing values, invalid email addresses, malformed date formats, broken foreign key references, negative quantities, and inconsistent text formatting. These inconsistencies were introduced to simulate real-world business data and to demonstrate the practical implementation of data preprocessing techniques.

The data cleaning process was implemented using the **Pandas** library in Python through the `clean_data.py` script. Each dataset was loaded into a separate DataFrame, where multiple validation

and preprocessing operations were performed before exporting the cleaned datasets for database loading.

### **Cleaning the Customers Dataset**

The Customers dataset was cleaned by removing duplicate records based on the Customer ID, handling missing email addresses, and validating email formats. Invalid email addresses were either corrected where possible or marked for removal depending on the severity of the inconsistency. Registration dates were converted into a consistent datetime format, and customer type values were standardized to ensure uniformity across all records.

### **Cleaning the Products Dataset**

The Products dataset was processed to remove duplicate product records and correct inconsistencies in product names. Extra spaces, irregular capitalization, and formatting issues were standardized to maintain consistency. Missing selling prices were handled appropriately, and numeric fields such as cost price, selling price, and stock quantity were converted to their respective data types for accurate calculations during analysis.

### **Cleaning the Orders Dataset**

The Orders dataset required extensive validation because it served as the central transaction table of the database. Duplicate order records were removed, null customer IDs were identified, and invalid date values were detected using Pandas datetime conversion techniques. Date values were standardized into the SQL-compatible format **YYYY-MM-DD HH:MM:SS**, ensuring compatibility with Microsoft SQL Server. Future dates and malformed date formats were identified and processed according to the project's data cleaning strategy.

### **Cleaning the Order Items Dataset**

The Order Items dataset was validated by checking the consistency of Order IDs and Product IDs against their corresponding master tables. Records containing broken foreign key references were identified and removed or corrected based on the implemented validation rules. Negative quantities, incorrect discount calculations, and inconsistent pricing values were also verified to maintain transactional accuracy.

### **Referential Integrity Validation**

One of the most important stages of the cleaning process involved validating referential integrity across all datasets. Relationships between Customers and Orders, as well as between Orders, Products, and Order Items, were verified to ensure that every foreign key referenced a valid primary key. Records violating these relationships were detected and appropriately handled before database import.

### Data Type Standardization

To ensure compatibility with SQL Server, all datasets underwent data type validation and conversion. Date columns were converted to standardized datetime formats, numeric columns were cast to appropriate integer or decimal types, and text columns were cleaned by removing unnecessary whitespace and correcting inconsistent formatting. This preprocessing ensured reliable storage and efficient querying within the relational database.

### Cleaned Dataset Output

After completing all validation and preprocessing operations, the cleaned datasets were exported as CSV files and stored in the **data/cleaned/** directory. These datasets contained consistent, validated, and analysis-ready records suitable for loading into Microsoft SQL Server.

| Cleaning Operation               | Purpose                                                        |
|----------------------------------|----------------------------------------------------------------|
| Duplicate Removal                | Eliminated repeated records from all datasets.                 |
| Missing Value Handling           | Processed null and incomplete records appropriately.           |
| Email Validation                 | Corrected or removed invalid email addresses.                  |
| Date Standardization             | Converted dates into SQL-compatible datetime format.           |
| Data Type Conversion             | Ensured consistency of numeric, text, and datetime fields.     |
| Referential Integrity Validation | Verified relationships between related tables.                 |
| Text Formatting                  | Standardized product names and other textual attributes.       |
| Numeric Validation               | Verified quantities, prices, discounts, and calculated values. |

The completion of the data cleaning phase significantly improved the quality, consistency, and reliability of the generated datasets. These cleaned datasets formed the foundation for the subsequent database implementation and SQL analytics phases, ensuring that business reports were generated from accurate and validated information.

### 3.3 Database Creation and Data Loading

After completing the data generation and preprocessing phases, the cleaned datasets were imported into Microsoft SQL Server for structured storage and analytical processing. A relational database was created to maintain data consistency, enforce relationships between tables, and enable efficient execution of complex SQL queries.

The database schema was implemented using the schema.sql script. The script created four relational tables—**Customers**, **Products**, **Orders**, and **Order\_Items**—along with appropriate primary key and foreign key constraints. These constraints ensured that each record was uniquely identifiable and that relationships between tables remained valid throughout the database.

The implementation began by creating a dedicated database for the project. Once the database was created, the required tables were defined with suitable data types for each attribute. Primary keys were assigned to uniquely identify records, while foreign keys established relationships between the transaction tables and their corresponding master tables.

**Database Tables**

The relational database contains the following tables:

- **Customers** – Stores customer profile information.
- **Products** – Stores product details and pricing information.
- **Orders** – Stores order-level transaction details.
- **Order\_Items** – Stores product-level information for each order.

The relationship between these tables enables efficient execution of JOIN operations for generating business reports and analytical insights.

**Primary Key Constraints**

Primary keys were implemented to uniquely identify each record in the database.

| Table       | Primary Key |
|-------------|-------------|
| Customers   | Customer_ID |
| Products    | Product_ID  |
| Orders      | Order_ID    |
| Order_Items | Item_ID     |

These primary keys prevent duplicate records and improve query performance through indexing.

**Foreign Key Relationships**

Foreign key constraints were implemented to preserve referential integrity between related tables.

**Child Table Foreign Key Parent Table**

|        |             |           |
|--------|-------------|-----------|
| Orders | Customer_ID | Customers |
|--------|-------------|-----------|

### **Child Table Foreign Key Parent Table**

Order\_Items Order\_ID Orders

Order\_Items Product\_ID Products

These relationships ensure that every order belongs to a valid customer and every order item references an existing order and product.

### **Data Import Process**

Once the database schema was created, the cleaned CSV files generated during the preprocessing phase were imported into SQL Server. The import process followed the parent-to-child relationship sequence to avoid foreign key violations.

The datasets were imported in the following order:

1. Customers
2. Products
3. Orders
4. Order\_Items

Importing the tables in this sequence ensured that all parent records existed before inserting dependent child records.

### **Data Validation After Import**

Following the successful import of all datasets, several validation checks were performed to verify data integrity.

The validation process included:

- Verification of total row counts in each table.
- Confirmation of successful primary key enforcement.
- Validation of foreign key relationships.
- Detection of orphan records.
- Verification of data types.
- Confirmation of successful loading without constraint violations.

These validation steps ensured that the database accurately reflected the cleaned datasets and was ready for analytical processing.

### **Database Integrity**



The implementation of relational constraints significantly improved the reliability of the database by preventing invalid data insertion and maintaining consistency between related tables. This structured database served as the foundation for all subsequent SQL analytics, customer segmentation, cohort analysis, and reporting operations.

By storing the cleaned data within Microsoft SQL Server, the project leveraged the advantages of a relational database management system, including efficient querying, optimized indexing, robust constraint enforcement, and support for advanced SQL features such as aggregate functions, Common Table Expressions (CTEs), and window functions. This database implementation enabled accurate business analysis while ensuring high data quality and consistency throughout the project.

### **3.4 SQL Analytics**

Once the cleaned datasets were successfully loaded into Microsoft SQL Server, advanced SQL queries were executed to extract meaningful business insights from the e-commerce data. SQL analytics formed the core of the project by transforming raw transactional data into valuable information that supports business decision-making. Various SQL concepts such as JOIN operations, aggregate functions, GROUP BY clauses, Common Table Expressions (CTEs), window functions, and ranking functions were used to perform comprehensive data analysis.

#### **Revenue Analysis**

Revenue analysis was performed to evaluate the overall sales performance of the e-commerce platform. The total revenue generated by each customer, product category, and month was calculated using aggregate functions. The revenue for each order item was determined by multiplying the unit price by the quantity purchased after applying the applicable discount percentage.

These analyses helped identify high-value customers, profitable product categories, and monthly sales trends.

#### **Join Operations**

Since the project data was distributed across multiple relational tables, SQL JOIN operations were extensively used to combine related information. INNER JOIN statements were implemented to connect Customers, Orders, Products, and Order\_Items tables through their respective primary and foreign key relationships.

The use of JOIN operations enabled the generation of consolidated reports containing customer details, purchased products, order information, pricing, discounts, and revenue within a single query result.

## **Aggregate Functions**

Aggregate functions were used to summarize transactional data and generate meaningful business metrics. Functions such as **SUM()**, **COUNT()**, **AVG()**, **MIN()**, and **MAX()** were applied to calculate various performance indicators.

Examples of analytical metrics generated include:

- Total revenue generated
- Number of orders placed
- Total quantity sold
- Average order value
- Average customer spending
- Highest and lowest transaction values

These aggregated results provided a clear understanding of business performance at different levels.

## **Grouping and Categorization**

The **GROUP BY** clause was used to categorize records according to customers, product categories, months, regions, and customer types. Grouping enabled the calculation of summarized statistics for each business dimension, allowing easier comparison of performance across different segments.

For example, monthly revenue reports grouped transactions by order month, while customer-level reports grouped transactions by Customer ID to calculate lifetime spending.

## **Product Performance Analysis**

Product performance analysis identified the most successful products based on sales volume and generated revenue. SQL queries calculated the total quantity sold and total revenue earned by each product.

This analysis helped determine:

- Best-selling products
- Highest revenue-generating products
- Product category performance
- Customer purchasing preferences

These insights can support inventory planning and product management decisions.

## **Customer Revenue Analysis**

Customer revenue analysis calculated the lifetime value of each customer by summing the revenue generated from all completed purchases. Customers were ranked according to their total spending, allowing the identification of high-value customers who contribute significantly to overall business revenue.

This information is particularly useful for customer loyalty programs and targeted marketing strategies.

## **Average Order Value (AOV)**

Average Order Value (AOV) was calculated by dividing the total revenue by the total number of completed orders. The analysis was further extended to compare AOV across different customer segments such as Regular, Premium, and VIP customers.

AOV serves as an important business metric for evaluating customer purchasing behavior and measuring the effectiveness of promotional strategies.

## **Monthly Revenue Trends**

Monthly sales analysis was performed by grouping transactions according to the order month. This analysis enabled visualization of revenue fluctuations over time and helped identify seasonal sales patterns and business growth trends.

The monthly reports can assist management in forecasting future sales and planning marketing campaigns more effectively.

## **Business Insights Generated**

The SQL analytics module successfully generated several valuable business insights, including:

- Total revenue generated by the business.
- Revenue contribution of each customer.
- Revenue generated by different product categories.
- Best-selling products based on quantity sold.
- Highest revenue-generating products.
- Monthly sales trends.
- Average Order Value (AOV).
- Customer purchase behavior.
- Product category performance.

- Regional sales distribution.

The implementation of SQL analytics demonstrated the effectiveness of relational databases in transforming transactional data into actionable business intelligence. Through the use of JOIN operations, aggregate functions, and grouping techniques, the project produced comprehensive analytical reports that support strategic decision-making. This phase established the foundation for the advanced analyses presented in the following sections, including window functions, customer segmentation, cohort analysis, and retention measurement.

### **3.5 Advanced SQL Analytics (Window Functions and Common Table Expressions)**

In addition to basic SQL queries, the project implemented advanced analytical techniques using **Window Functions** and **Common Table Expressions (CTEs)**. These SQL features enabled complex business calculations without modifying the underlying data and significantly improved the readability and efficiency of analytical queries.

Window functions perform calculations across a set of related rows while preserving the individual rows in the result set. Unlike aggregate functions, window functions do not collapse multiple rows into a single output, making them ideal for ranking, cumulative calculations, and trend analysis. Common Table Expressions (CTEs) were used to divide complex SQL queries into smaller, reusable logical components, resulting in cleaner and more maintainable query structures.

#### **Customer Ranking**

Customer ranking was implemented to identify the highest-value customers based on their total spending. The **RANK()** and **DENSE\_RANK()** window functions were used to assign rankings according to customer lifetime value.

This analysis enabled the identification of premium customers who contribute the highest revenue to the business. Ranking customers also supports loyalty programs, personalized marketing campaigns, and customer relationship management.

#### **Running Revenue Calculation**

The project calculated cumulative revenue over time using the **SUM() OVER()** window function. Running totals provide a continuous view of business growth by showing the accumulated revenue after each transaction or reporting period.

This analysis is useful for monitoring sales performance and understanding revenue progression throughout the financial year.

#### **Moving Average Analysis**

To reduce fluctuations in monthly sales data, the **AVG() OVER()** window function was used to calculate moving averages. The moving average smooths short-term variations and helps identify long-term revenue trends.

This analytical technique assists business managers in understanding seasonal demand and forecasting future sales more accurately.

### **Common Table Expressions (CTEs)**

Common Table Expressions were extensively used to simplify complex SQL queries. Instead of writing deeply nested subqueries, intermediate calculations were stored temporarily within CTEs and reused throughout the query.

CTEs improved query readability while making the implementation of multi-step business calculations more organized and easier to maintain.

Examples of CTE-based analyses performed in the project include:

- Monthly revenue calculation
- Customer lifetime value computation
- Product revenue aggregation
- Revenue growth analysis
- Customer segmentation metrics

### **Monthly Revenue Growth**

Monthly revenue growth was calculated by first generating monthly revenue using a Common Table Expression. Window functions were then applied to compare each month's revenue with the previous month, allowing the calculation of month-over-month growth.

This analysis provides valuable insight into business performance trends and highlights periods of increasing or declining sales activity.

### **Benefits of Advanced SQL Analytics**

The implementation of window functions and CTEs provided several advantages throughout the project:

- Simplified complex analytical queries.
- Improved query readability and maintainability.
- Eliminated unnecessary nested subqueries.
- Enabled ranking and comparative analysis.

- Supported cumulative calculations and trend analysis.
- Improved performance for analytical reporting.
- Generated business insights without altering the original datasets.

The use of advanced SQL features significantly enhanced the analytical capabilities of the system. Window functions enabled efficient ranking, running totals, and moving average calculations, while Common Table Expressions simplified the implementation of multi-step business logic. Together, these techniques allowed the project to generate sophisticated business insights that would be difficult to obtain using basic SQL queries alone, demonstrating practical applications of advanced SQL in modern data analytics.

### **3.6 Cohort Analysis and Customer Segmentation**

Understanding customer purchasing behavior is essential for improving customer retention, increasing revenue, and developing effective marketing strategies. In this project, advanced SQL techniques were used to perform cohort analysis, customer retention analysis, purchase frequency segmentation, spend-based segmentation, and RFM (Recency, Frequency, Monetary) analysis. These analytical techniques provided valuable insights into customer engagement and long-term business performance.

#### **Cohort Analysis**

Cohort analysis groups customers based on a shared characteristic, such as the month of their first purchase. In this project, customers were assigned to cohorts according to the month in which they placed their first order. This approach enabled the comparison of customer behavior across different acquisition periods.

Monthly cohorts were analyzed to determine how many customers continued making purchases in subsequent months. This analysis helped identify trends in customer engagement and highlighted the effectiveness of customer retention over time.

#### **Customer Retention Analysis**

Customer retention analysis measured the percentage of customers who returned to place additional orders after their initial purchase. Monthly retention rates were calculated for each cohort by comparing the number of active customers in each month with the total number of customers in the original cohort.

Higher retention rates indicate stronger customer loyalty, while declining retention rates may suggest reduced customer engagement or ineffective retention strategies. These insights enable businesses to evaluate customer satisfaction and improve long-term customer relationships.

## Churn Analysis

Customers who did not make any additional purchases after their initial transaction were classified as churned customers. By identifying churned customers, the business can develop targeted promotional campaigns and personalized offers to encourage repeat purchases.

The analysis differentiated customers into two categories:

- **Repeat Customers** – Customers who placed more than one order.
- **Churned Customers** – Customers who placed only a single order during the analysis period.

Monitoring customer churn helps organizations identify potential revenue loss and implement strategies to improve customer retention.

## Purchase Frequency Segmentation

Customers were segmented according to the number of orders they placed during the analysis period. This segmentation provides a better understanding of purchasing behavior and customer engagement.

The customer groups were classified as follows:

| Customer Segment | Purchase Frequency |
|------------------|--------------------|
|------------------|--------------------|

|                   |                      |
|-------------------|----------------------|
| One-Time Customer | Exactly one purchase |
|-------------------|----------------------|

|                     |                       |
|---------------------|-----------------------|
| Occasional Customer | Two to five purchases |
|---------------------|-----------------------|

|                |                          |
|----------------|--------------------------|
| Loyal Customer | More than five purchases |
|----------------|--------------------------|

This classification helps businesses identify their most engaged customers and design loyalty programs accordingly.

## Spend-Based Segmentation

Customers were also categorized according to their total purchase value. The cumulative spending of each customer was calculated using SQL aggregate functions and grouped into different spending tiers.

The segmentation categories included:

| Spend Tier | Description |
|------------|-------------|
|------------|-------------|

|           |                                                |
|-----------|------------------------------------------------|
| Low Value | Customers with relatively low overall spending |
|-----------|------------------------------------------------|

|              |                                             |
|--------------|---------------------------------------------|
| Medium Value | Customers with moderate purchasing activity |
|--------------|---------------------------------------------|

| Spend Tier | Description |
|------------|-------------|
|------------|-------------|

|            |                                            |
|------------|--------------------------------------------|
| High Value | Customers contributing the highest revenue |
|------------|--------------------------------------------|

Spend-based segmentation assists businesses in identifying high-value customers for premium services, personalized recommendations, and exclusive promotional campaigns.

### RFM Analysis

The project implemented an RFM (Recency, Frequency, Monetary) model to evaluate customer value using three key performance indicators:

- **Recency (R):** Number of days since the customer's most recent purchase.
- **Frequency (F):** Total number of orders placed by the customer.
- **Monetary (M):** Total amount spent by the customer across all purchases.

Each customer was evaluated based on these three metrics to identify purchasing behavior and overall customer value. Customers with recent purchases, high purchase frequency, and high monetary value were considered the most valuable customers for the business.

RFM analysis enables organizations to:

- Identify loyal customers.
- Detect inactive customers.
- Prioritize marketing campaigns.
- Improve customer retention strategies.
- Increase customer lifetime value.

### Business Benefits

The implementation of cohort analysis and customer segmentation provided several important business insights:

- Identification of high-value customers.
- Measurement of customer retention performance.
- Detection of customer churn patterns.
- Evaluation of purchasing behavior across customer groups.
- Improved understanding of customer lifetime value.
- Support for targeted marketing and loyalty programs.



- Better strategic decision-making through customer analytics.

By combining cohort analysis, retention measurement, purchase frequency segmentation, spend-based categorization, and RFM analysis, the project successfully transformed transactional data into meaningful customer intelligence. These analytical techniques provide businesses with a deeper understanding of customer behavior, enabling data-driven decisions that improve customer satisfaction, strengthen retention strategies, and maximize long-term profitability.

### 3.7 Command-Line Reporting Tool and Error Handling

The final implementation phase of the project involved the development of a Python-based Command-Line Interface (CLI) reporting tool. The primary objective of this module was to provide users with a simple and interactive method of accessing business reports directly from the Microsoft SQL Server database without manually executing SQL queries.

The reporting tool was developed using Python and connected to Microsoft SQL Server through the **PyODBC** library. Based on the command-line arguments supplied by the user, the application dynamically executed predefined SQL queries and displayed the results in a structured tabular format using the **Tabulate** library.

#### Command-Line Reporting Tool

The CLI reporting tool accepts user-defined parameters that specify the type of report to be generated. After validating the input, the application establishes a connection with the SQL Server database, executes the corresponding SQL query, retrieves the result set, and displays the output in a well-formatted table.

Examples of reports supported by the system include:

- Customer Revenue Report
- Top Customers Report
- Monthly Revenue Report
- Product Performance Report
- Customer Segmentation Report
- Cohort Retention Report
- Average Order Value (AOV) Report

This approach allows users to generate multiple business reports without modifying the underlying source code.

#### Dynamic SQL Execution

The reporting module was designed to execute SQL queries dynamically according to the selected report type. Each command entered by the user is mapped to a predefined SQL query stored within the application. This modular design improves code readability, simplifies maintenance, and allows new reports to be added with minimal changes.

Dynamic query execution also reduces manual effort while providing quick access to business insights generated from the database.

### **Database Connectivity**

The CLI application communicates with Microsoft SQL Server using the PyODBC library. Before executing any SQL query, the application attempts to establish a secure database connection. Once the query execution is completed, the connection is properly closed to prevent unnecessary resource usage.

Maintaining reliable database connectivity ensures accurate report generation and efficient use of system resources.

### **Report Formatting**

The query results are displayed using the **Tabulate** library, which presents data in a clean, readable table format. Proper formatting improves readability and enables users to interpret business metrics more efficiently.

Formatted reports include column headings, aligned values, and structured layouts that resemble professional business reports.

### **Input Validation**

To improve system reliability, the application validates all command-line arguments before executing any database operation. Invalid report names, missing parameters, or unsupported commands are detected immediately, and meaningful error messages are displayed to guide the user.

Input validation prevents unnecessary query execution and enhances the overall user experience.

### **Error Handling**

Robust exception handling was implemented throughout the reporting tool to ensure stable system performance. The application gracefully handles runtime exceptions without terminating unexpectedly.

The implemented error-handling mechanisms include:

- Invalid command-line arguments.
- Database connection failures.

- SQL query execution errors.
- Missing database tables.
- Empty result sets.
- Invalid user inputs.
- Unexpected runtime exceptions.

Whenever an error occurs, the application displays informative messages that help users identify and resolve the problem quickly.

### **Edge Case Testing**

The reporting system was tested under multiple edge-case scenarios to verify its robustness and reliability. Testing included situations where the database contained limited or abnormal data, ensuring that the application continued to operate correctly under exceptional conditions.

The following scenarios were considered during testing:

- Empty database tables.
- Single customer with no orders.
- Orders containing zero revenue.
- Future transaction dates.
- Missing foreign key references.
- Invalid report names.
- Database connection interruptions.

The successful handling of these edge cases demonstrates the reliability and stability of the reporting system.

### **Summary**

The Command-Line Reporting Tool provides an efficient interface for generating business reports directly from the SQL Server database. Through dynamic SQL execution, structured report formatting, comprehensive input validation, and robust error handling, the application delivers accurate analytical results while maintaining high system reliability. The implementation of edge-case testing further strengthens the quality of the project by ensuring that the system performs consistently under both normal and exceptional operating conditions.

# CHAPTER 4: RESULTS AND DISCUSSION

## 4.1 Experimental Results

After completing the implementation of the E-Commerce Order Analytics System, the developed solution was successfully executed on the generated datasets. The system performed all stages of the analytics pipeline, including dataset generation, data cleaning, database creation, SQL analytics, customer segmentation, cohort analysis, and report generation through the command-line interface.

The raw datasets intentionally contained inconsistencies such as duplicate records, missing values, invalid email addresses, malformed date formats, negative quantities, and broken foreign key references. These inconsistencies simulated real-world business data and allowed the effectiveness of the data cleaning process to be evaluated.

During preprocessing, the system successfully identified and handled duplicate records, standardized date formats, validated email addresses, corrected inconsistent text formatting, removed invalid references, and ensured referential integrity between related tables. The cleaned datasets were then imported into Microsoft SQL Server without violating relational constraints, confirming the correctness of the preprocessing phase.

The SQL analytics module generated several business reports that provided meaningful insights into customer purchasing behavior, product performance, and revenue trends. Revenue calculations, customer lifetime value analysis, average order value, top-selling products, and monthly sales reports were successfully produced using SQL joins, aggregate functions, Common Table Expressions (CTEs), and window functions.

Customer segmentation techniques further classified customers according to purchase frequency, spending behavior, and RFM (Recency, Frequency, Monetary) metrics. Cohort analysis measured customer retention across different acquisition periods, enabling the identification of repeat customers and customer churn patterns.

The Python-based Command-Line Interface (CLI) successfully connected to Microsoft SQL Server using the PyODBC library and dynamically generated business reports based on user-selected options. Report outputs were displayed in a structured tabular format, providing a simple and user-friendly reporting experience.

The system also demonstrated robust performance during edge-case testing. It correctly handled invalid user inputs, database connection errors, empty result sets, malformed records, and other exceptional conditions without unexpected termination. This verified the reliability and stability of the overall system.

The successful execution of all project modules confirmed that the proposed analytics system effectively integrates Python and SQL to perform end-to-end processing of e-commerce data. The

generated business reports provide meaningful insights that can support strategic decision-making, customer relationship management, sales analysis, and business performance evaluation.

### Summary of Experimental Results

| Module                  | Outcome                                                                                          |
|-------------------------|--------------------------------------------------------------------------------------------------|
| Dataset Generation      | Successfully generated realistic e-commerce datasets with controlled inconsistencies.            |
| Data Cleaning           | Removed duplicates, handled missing values, standardized formats, and validated relationships.   |
| Database Implementation | Successfully created SQL Server database with primary and foreign key constraints.               |
| SQL Analytics           | Generated revenue reports, sales analysis, customer insights, and product performance metrics.   |
| Window Functions & CTEs | Successfully implemented customer ranking, running totals, and monthly growth analysis.          |
| Cohort Analysis         | Calculated customer retention and identified repeat and churned customers.                       |
| Customer Segmentation   | Successfully classified customers using purchase frequency, spending behavior, and RFM analysis. |
| CLI Reporting Tool      | Generated dynamic business reports with proper formatting and input validation.                  |
| Error Handling          | Successfully handled invalid inputs, empty results, and database exceptions.                     |

The experimental results demonstrate that the proposed system fulfills all the objectives defined at the beginning of the project. The integration of Python for data processing and Microsoft SQL Server for relational analytics resulted in an efficient, scalable, and reliable e-commerce analytics solution capable of transforming raw transactional data into valuable business intelligence.

## 4.2 Analysis of Results

The implementation of the E-Commerce Order Analytics System successfully demonstrated the complete workflow of a modern data analytics pipeline, beginning with raw data generation and ending with business intelligence reporting. The analytical results obtained from the system

indicate that integrating Python with Microsoft SQL Server provides an efficient solution for processing and analyzing large volumes of e-commerce transaction data.

The data generation module successfully created realistic datasets containing both valid records and intentionally introduced inconsistencies. By including duplicate entries, missing values, malformed date formats, invalid email addresses, and broken foreign key references, the generated data closely resembled real-world business data. This allowed the subsequent data cleaning phase to be evaluated under practical conditions rather than ideal scenarios.

The preprocessing module significantly improved data quality by identifying and correcting inconsistencies across all datasets. Duplicate records were removed, missing values were handled appropriately, invalid data formats were standardized, and referential integrity was restored between related tables. These preprocessing operations ensured that the datasets loaded into Microsoft SQL Server were accurate, consistent, and suitable for advanced analytical processing.

The SQL analytics component successfully transformed transactional data into meaningful business information. Revenue calculations, customer-wise sales analysis, monthly revenue reports, product performance evaluation, and average order value calculations provided valuable insights into the operational performance of the simulated e-commerce business. The use of JOIN operations and aggregate functions enabled comprehensive reporting across multiple relational tables.

Advanced SQL techniques further enhanced the analytical capabilities of the system. Window functions enabled customer ranking, running revenue calculations, and moving average analysis without modifying the original datasets. Common Table Expressions (CTEs) simplified complex analytical queries by dividing multi-step calculations into manageable components, improving both readability and maintainability.

Customer segmentation produced valuable business insights by categorizing customers according to purchase frequency, total spending, and RFM (Recency, Frequency, Monetary) metrics. This analysis made it possible to identify loyal customers, occasional buyers, one-time purchasers, and high-value customers. Such information can be used by organizations to develop targeted marketing campaigns, customer retention strategies, and loyalty programs.

Cohort analysis provided an additional perspective on customer behavior by measuring customer retention across different acquisition periods. The analysis highlighted patterns of repeat purchases and customer churn, enabling businesses to evaluate the long-term effectiveness of customer engagement strategies and identify opportunities for improving customer lifetime value.

The Command-Line Interface (CLI) reporting tool simplified access to business reports by allowing users to generate analytical summaries through predefined commands. The integration of Python with Microsoft SQL Server enabled dynamic report generation, while structured output formatting improved report readability. Input validation and exception handling further enhanced the reliability of the application.

Overall, the analytical results confirm that the developed system effectively converts raw transactional data into actionable business intelligence. The generated reports provide meaningful insights that support informed decision-making in areas such as sales performance evaluation, inventory planning, customer relationship management, revenue optimization, and business growth analysis. The modular architecture adopted in the project also ensures that additional analytical features can be incorporated in future without major modifications to the existing system.

The successful implementation of the proposed analytics pipeline demonstrates the practical application of Python, SQL, and relational database concepts in solving real-world business problems. The project satisfies all predefined objectives and serves as an effective example of end-to-end data engineering and business analytics using industry-standard technologies.

### **4.3 Advantages of the Proposed System**

The E-Commerce Order Analytics System offers several advantages by combining Python-based data processing with Microsoft SQL Server for relational database management and business analytics. The system automates the complete workflow from dataset generation to analytical reporting, reducing manual effort while improving data accuracy and operational efficiency.

One of the major advantages of the proposed system is its ability to simulate real-world e-commerce data. The generated datasets include intentional inconsistencies such as duplicate records, missing values, invalid email addresses, malformed dates, and broken foreign key references. This enables realistic testing of data cleaning and validation techniques, making the project suitable for learning practical data engineering concepts.

The implementation of automated data cleaning using the Pandas library significantly improves data quality. Duplicate records are removed, missing values are handled, invalid formats are standardized, and referential integrity is verified before the data is loaded into the database. This ensures that the analytical results are based on consistent and reliable information.

Another important advantage is the use of a relational database implemented in Microsoft SQL Server. The database schema incorporates primary keys, foreign keys, and integrity constraints, ensuring data consistency while enabling efficient execution of complex SQL queries. The structured storage also minimizes redundancy and supports scalable data management.

The project demonstrates the practical use of advanced SQL features such as JOIN operations, aggregate functions, Common Table Expressions (CTEs), window functions, customer ranking, running totals, and cohort analysis. These analytical techniques transform transactional data into meaningful business insights that support data-driven decision-making.

The customer segmentation module provides valuable information about customer behavior by classifying customers based on purchase frequency, spending patterns, and RFM (Recency,

Frequency, Monetary) analysis. Such insights can assist businesses in identifying loyal customers, improving customer retention, and designing targeted marketing campaigns.

The Python-based Command-Line Interface (CLI) further enhances usability by allowing users to generate business reports dynamically through simple commands. Reports are displayed in a structured tabular format, making analytical information easy to interpret without requiring users to manually execute SQL queries.

The system also incorporates comprehensive input validation and exception handling mechanisms. Invalid user inputs, database connection failures, and empty result sets are handled gracefully, improving the overall reliability and robustness of the application.

From a software engineering perspective, the modular architecture adopted in the project improves maintainability and scalability. Separate modules are used for data generation, data cleaning, database management, SQL analytics, and report generation. This modular design allows new functionalities to be added with minimal modifications to the existing codebase.

Overall, the proposed system provides an effective, scalable, and reliable solution for e-commerce data analytics. It integrates modern programming techniques, relational database concepts, and business intelligence methodologies to automate data processing and generate valuable insights that can support strategic business decisions.

### **Key Advantages**

- Automates the complete data analytics workflow from data generation to reporting.
- Simulates realistic e-commerce datasets with controlled inconsistencies.
- Improves data quality through automated cleaning and validation.
- Maintains data integrity using relational database constraints.
- Supports advanced SQL analytics using JOINS, CTEs, and window functions.
- Generates meaningful business insights through customer segmentation and cohort analysis.
- Provides dynamic report generation through a Python-based CLI.
- Handles invalid inputs and runtime exceptions effectively.
- Follows a modular and scalable software architecture.
- Demonstrates practical implementation of Python, SQL, and data analytics concepts suitable for real-world applications.



## 4.4 Limitations of the Proposed System

Although the E-Commerce Order Analytics System successfully achieves its intended objectives, certain limitations exist due to the scope of the project and the resources available during development. These limitations provide opportunities for future enhancements and demonstrate areas where the system can be further improved to support real-world enterprise applications.

One of the primary limitations of the project is that the datasets are synthetically generated using Python libraries such as Faker and Random. While the generated data closely resembles real-world e-commerce transactions, it does not fully capture the complexity, unpredictability, and scale of production-level business data collected from actual customers.

The current implementation is designed to process datasets stored in CSV format before loading them into Microsoft SQL Server. As a result, the system does not support real-time data ingestion or continuous data streaming from live e-commerce platforms. Organizations handling high-volume transactions would require integration with streaming technologies and automated ETL pipelines.

The reporting component of the project is implemented as a Command-Line Interface (CLI). Although the CLI effectively generates business reports, it does not provide graphical dashboards, interactive visualizations, or web-based reporting features. Business users often prefer visual dashboards for faster interpretation of analytical results.

Another limitation is that the project focuses primarily on descriptive analytics. It analyzes historical transaction data to identify business trends but does not include predictive or prescriptive analytics. Machine learning techniques for demand forecasting, customer purchase prediction, recommendation systems, and sales forecasting have not been incorporated into the current implementation.

The database implementation is designed for educational purposes using a moderate-sized dataset. Although Microsoft SQL Server can efficiently manage significantly larger datasets, the project has not been benchmarked for enterprise-scale data volumes involving millions of customer records and transactions.

The customer segmentation module currently relies on rule-based classification using purchase frequency, spending patterns, and RFM (Recency, Frequency, Monetary) analysis. More advanced segmentation techniques based on clustering algorithms, customer lifetime prediction, or behavioral analytics could provide deeper business insights.

In addition, the project does not include authentication, authorization, user management, or role-based access control. Such security mechanisms are essential for production environments where sensitive customer and transactional data must be protected against unauthorized access.

Despite these limitations, the proposed system successfully demonstrates the practical implementation of Python, Microsoft SQL Server, and advanced SQL analytics for solving real-

world data processing problems. The modular architecture adopted in the project allows additional features such as web dashboards, cloud deployment, machine learning models, real-time analytics, and automated data pipelines to be integrated in future versions without requiring significant changes to the existing system.

### **Summary of Limitations**

- Uses synthetically generated data instead of real production datasets.
- Does not support real-time data ingestion or streaming analytics.
- Provides a command-line interface instead of a graphical dashboard.
- Focuses on descriptive analytics rather than predictive analytics.
- Not evaluated using enterprise-scale datasets.
- Uses rule-based customer segmentation instead of machine learning techniques.
- Does not include authentication or role-based access control.
- Deployment is limited to a local development environment.

These limitations do not affect the successful achievement of the project objectives but instead identify potential directions for future enhancement. Incorporating these improvements would increase the scalability, usability, and analytical capabilities of the system, making it suitable for deployment in real-world business environments.

# CHAPTER 5: CONCLUSION AND FUTURE SCOPE

## 5.1 Conclusion

The **E-Commerce Order Analytics System Using Python and SQL** successfully achieved the objective of designing and implementing a complete end-to-end data analytics pipeline for processing and analyzing e-commerce order data. The project integrated Python for dataset generation, data preprocessing, database connectivity, and report generation with Microsoft SQL Server for structured data storage and advanced SQL analytics.

The project began with the generation of realistic e-commerce datasets containing customers, products, orders, and order items. To simulate practical business scenarios, controlled inconsistencies such as duplicate records, missing values, invalid email addresses, malformed date formats, negative quantities, and broken foreign key references were intentionally introduced. This approach enabled the implementation of realistic data cleaning and validation techniques using the Pandas library.

The preprocessing phase successfully improved data quality by removing duplicate records, handling missing values, standardizing formats, validating referential integrity, and ensuring compatibility with the relational database. The cleaned datasets were then imported into Microsoft SQL Server, where an efficient relational database schema was created using primary keys, foreign keys, and integrity constraints.

Advanced SQL queries were implemented to generate meaningful business insights from transactional data. SQL JOIN operations, aggregate functions, Common Table Expressions (CTEs), and window functions were used to analyze revenue trends, product performance, customer lifetime value, monthly sales, and average order value. Cohort analysis and RFM-based customer segmentation further enhanced the analytical capabilities of the system by providing valuable information about customer retention, purchasing behavior, and customer loyalty.

The Python-based Command-Line Interface (CLI) enabled users to generate business reports dynamically by executing predefined SQL queries. The application successfully incorporated database connectivity, structured report formatting, input validation, and exception handling to ensure reliable and user-friendly operation.

Overall, the project demonstrates the practical application of Python, Pandas, Microsoft SQL Server, and advanced SQL techniques in solving real-world data engineering and business analytics problems. The developed system successfully transforms raw transactional data into meaningful business intelligence that can support data-driven decision-making, customer relationship management, sales analysis, and strategic business planning.

The successful completion of this project has strengthened practical knowledge of database management, data preprocessing, SQL analytics, customer segmentation, and software

development while providing valuable experience in designing and implementing an end-to-end analytics solution following industry-standard practices.

## 5.2 Future Scope

Although the proposed E-Commerce Order Analytics System successfully meets its objectives, there are several opportunities for further enhancement that can improve its functionality, scalability, and practical applicability in real-world business environments.

One potential improvement is the integration of real-time data processing. Instead of relying on CSV files, the system can be connected directly to live e-commerce platforms through APIs or streaming technologies such as Apache Kafka. This would enable continuous monitoring of customer transactions and provide businesses with real-time analytical insights.

The current Command-Line Interface (CLI) can also be extended into a web-based application using frameworks such as Flask or Django. A graphical user interface (GUI) would provide a more interactive and user-friendly experience, allowing business users to generate reports, filter data, and analyze trends without requiring command-line knowledge.

Another important enhancement is the development of interactive dashboards using visualization tools such as Power BI, Tableau, or Python libraries including Plotly and Dash. These dashboards could present revenue trends, customer behavior, product performance, and sales statistics through charts, graphs, and key performance indicators (KPIs), making business insights easier to understand.

The project can also be expanded by incorporating machine learning techniques for predictive analytics. Models could be developed to forecast sales, predict customer churn, estimate customer lifetime value, recommend products, and identify purchasing patterns. Such capabilities would allow businesses to move beyond descriptive analytics and make proactive, data-driven decisions.

Cloud deployment represents another promising area for future development. The system could be hosted on cloud platforms such as Microsoft Azure, Amazon Web Services (AWS), or Google Cloud Platform (GCP). Cloud-based deployment would improve scalability, availability, and accessibility while supporting larger datasets and multiple concurrent users.

Additional security features can also be incorporated into the system. Authentication, role-based access control, encrypted database connections, and secure user management would make the application suitable for enterprise environments where customer and transactional data must be protected.

Future versions may also include automated ETL (Extract, Transform, Load) pipelines that periodically collect data from multiple sources, perform validation and transformation, and update

the database without manual intervention. This would reduce operational effort and improve data freshness.

Furthermore, the analytical capabilities of the project can be expanded by integrating inventory management, supplier analytics, shipping performance analysis, fraud detection, demand forecasting, and customer sentiment analysis. These additional modules would provide a more comprehensive business intelligence solution for modern e-commerce organizations.

In summary, the proposed system establishes a strong foundation for e-commerce analytics while offering numerous opportunities for future enhancement. By incorporating real-time processing, advanced visualization, cloud computing, machine learning, automation, and enterprise-grade security, the system can evolve into a complete business intelligence platform capable of supporting large-scale commercial operations and strategic decision-making.